

Module 5 — TypeScript

Time: 1.5 hours · Covers Practical 8 & 9 · CO3

Practical 8: Programming with different data structures and functions using TypeScript. Practical 9: Programming with classes and inheritance.

Part A — Why TypeScript? (concept, ~10 min)

JavaScript has no type checking — mistakes surface only at runtime:

```
function double(n) { return n * 2; }
double("5"); // JS: "55"? NaN? No warning until it breaks in production.
```

TypeScript is JavaScript **plus static types**. You annotate what a value *should* be, and the compiler catches mismatches **before** the code runs.

```
function double(n: number): number { return n * 2; }
double("5"); // ERROR at compile time: "5" is not a number
```

TypeScript compiles ("transpiles") down to plain JavaScript that Node and browsers run. React and Node both use it heavily — this is why it is in the syllabus.

Setup for this module

```
cd 05-typescript
npm install          # installs typescript + ts-node
npx tsc --noEmit     # type-check everything without producing files
npx ts-node src/01-basics.ts # run a file directly
```

Part B — Basic types & functions (P8, ~25 min)

```
// Primitive type annotations
let name: string = "Ada";
let age: number = 36;
let isStudent: boolean = true;

// Arrays
let scores: number[] = [90, 85, 77];

// A function with typed parameters and a typed return value
function add(a: number, b: number): number {
  return a + b;
}

// Optional (?) and default parameters
function greet(name: string, title?: string): string {
```

```
    return title ? `${title} ${name}` : `Hello ${name}`;
}
```

Type aliases and **interfaces** describe object shapes:

```
interface User {
  id: number;
  name: string;
  email?: string; // optional
}

function printUser(u: User): void {
  console.log(u.id, u.name);
}
```

See [src/01-basics.ts](#) .

Part C — Data structures (P8, ~25 min)

TypeScript adds types to the JavaScript structures you already know.

Arrays & tuples

```
const nums: number[] = [1, 2, 3];
const pair: [string, number] = ["age", 36]; // tuple: fixed length + types
```

Objects / records

```
const prices: Record<string, number> = { apple: 30, banana: 10 };
```

Map (keyed collection, any key type)

```
const scores = new Map<string, number>();
scores.set("Ada", 95);
scores.get("Ada"); // 95, typed as number | undefined
```

Set (unique values)

```
const tags = new Set<string>(["ts", "node", "ts"]); // "ts" stored once
```

Generics (reusable, type-safe containers)

```
// <T> is a placeholder for "whatever type the caller uses"
function first<T>(items: T[]): T | undefined {
  return items[0];
}
```

```
}  
first<number>([1, 2, 3]); // returns number  
first<string>(["a", "b"]); // returns string
```

Full, runnable examples: [src/02-data-structures.ts](#) .

Part D — Classes & inheritance (P9, ~25 min)

A **class** is a blueprint for objects: it bundles data (properties) and behavior (methods).

```
class Animal {  
  // access modifiers: public (default), private, protected, readonly  
  constructor(protected name: string, private sound: string) {}  
  
  makeSound(): string {  
    return `${this.name} says ${this.sound}`;  
  }  
}
```

Inheritance lets one class build on another with `extends` :

```
class Dog extends Animal {  
  constructor(name: string) {  
    super(name, "Woof"); // call the parent constructor  
  }  
  
  // OVERRIDE the parent method  
  makeSound(): string {  
    return `${this.name} barks: Woof!`;  
  }  
}
```

Key ideas demonstrated in the code:

- **super** — call the parent's constructor/methods.
- **Method overriding** — a child redefines a parent method (polymorphism).
- **Access modifiers** — `private` hides internals, `protected` shares with subclasses, `readonly` prevents reassignment.
- **Abstract classes** — a base that cannot be instantiated, forcing children to implement certain methods.
- **interface + implements** — a contract a class must fulfill.

Full example with an inheritance hierarchy: [src/03-classes-inheritance.ts](#) .

Summary

- TypeScript = JavaScript + **static types** → catch bugs at compile time.
- Type functions, arrays, tuples, `Map` , `Set` , `Record` , and use **generics** for reusable type-safe code.

- **Classes** bundle data + behavior; **inheritance** (`extends` , `super` , overriding) builds hierarchies; **access modifiers** control visibility.

Now do [practice.md](#) .